CS 2143 · OBJECT-ORIENTED PROGRAMMING

Streams, Friends & the Many Ways to Build an Object

Review worksheet — `operator<<` , `friend` , overloaded constructors, initializer lists

Name Gopina Ravi Date 09/07/26 Section _____

array[0] Fill in the Blank

one word / token per blank

1. To make `cout << f;` work for your own type, you overload `operator<<` . It returns ostream & so that `<<` calls can be chained.
2. A stream-insertion overload takes `std::ostream&` as its first parameter and a `const` reference to your class as its second.
3. `operator<<` is written as a non -member function because its left operand is a stream object, not an instance of your class.
4. The friend keyword lets a non-member function (or another class) reach the `private` members of a class.
5. A constructor callable with no arguments — because it takes none, or supplies a default for every parameter — is the default constructor.
6. Giving one class several constructors that differ in their parameter lists is an example of function overloading.
7. The comma-separated list written after a colon, between a constructor's signature and its `{ }` body, is the member-initializer list.
8. Data members are initialized in the order they are declared in the class — not the order they appear in the initializer list.
9. A `const` data member or a reference data member *must* be set in the member-initializer list, since it cannot be assigned in the constructor body.

Word bank ostream first non friend default overloading member initializer declared

array[1] Find the Error

continued on back

CASE A COMPILES? Y / (N)

```
class Fraction {
    int num, den;
public:
    Fraction(int n, int d)
      : num(n), den(d) {}

    ostream& operator<<(ostream& os) {
        return os << num << "/" << den;
    }
};

int main() {
    Fraction f(1, 2);
    cout << f;          // line 13
}
```

Why does `cout << f;` (line 13) not match this overload? What signature would?

Since cout << f; is a member function, its left operand is the Fraction object, not the stream - it defines f << os, not os << f. The signature ostream& operator<< (ostream& os, const Fraction& f); would work.

CASE B COMPILES? Y / (N)

```
class Fraction {
    int num, den;
public:
    Fraction(int n, int d)
      : num(n), den(d) {}
};

ostream& operator<<(ostream& os,
                    const Fraction& f) {
    return os << f.num
              << "/" << f.den;
}
```

Which two expressions are rejected, and why? Write the one line to add to the class.

f.num and f.den would be rejected because they are private members, and this operator<< is a non-member function with no special access. The line added to the class would be:

friend ostream& operator<< (ostream& os, const Fraction& f);

treams, Friends & the Many Ways to Build an Object

Review worksheet — `operator<<` , `friend` , overloaded constructors, initializer lists

array[1] Find the Error

CASE C

COMPILES? Y / (N)

```
class Circle {
    const double PI;
    double r;
public:
    Circle(double radius) {
        PI = 3.14159;
        r  = radius;
    }
};
```

Name the compile error and the exact line. Rewrite the constructor so it is correct.

A const data member cannot be assigned in the constructor body; the error is on the line `PI = 3.14159;` The correct constructor would be: `Circle(double radius) : PI(3.14159), r(radius) {}`

CASE D

COMPILES? Y / (N)

```
class Fraction {
    int num, den;
public:
    Fraction(int n) : num(n), den(1) {}
    Fraction(int n, int d = 1)
      : num(n), den(d) {}
};

Fraction b(3);   // rejected
```

`Fraction b(3);` will not compile. Why? Give two different one-line fixes (one of them a single constructor with default arguments).

Since the call is ambiguous, `b(3)` matches both `Fraction(int n)` and `Fraction(int n, int d=1)` equally well and compiler cannot choose between them.

COMPILES? (Y) / N

Fix 1: Delete `Fraction(int n)` and keep only `Fraction(int n, int d = 1) : num(n), den(d) {}`

Fix 2: Remove the default from the two-arg constructor: `Fraction(int n, int d) : num(n), den(d) {}`

CASE E

```
class Point {
    int x, y;
public:
    Point(int x, int y) { x = x; y = y; }
    friend ostream& operator<<(ostream& os, const Point& p);
};
ostream& operator<<(ostream& os, const Point& p) {
    return os << "(" << p.x << ", " << p.y << ")";
}
int main() { Point p(3, 4);  cout << p; }   // prints garbage
```

The `friend` overload is fine, yet this prints garbage. What is wrong inside the constructor, and how does a member initializer list fix it in one stroke?

The parameters x and y shadow the members, so `x = x;` and `y = y;` just assign each parameter to itself — the real members stay uninitialized. A member initializer list fixes it in one stroke: `Point(int x, int y) : x(x), y(y) {}` correctly initializes the members from the parameters.

array[2] Short Answer

full sentences

1. Explain why `operator<<` for a user-defined type is almost always a non-member function. Refer to what the left operand of `cout << obj` actually is.

   The left operand of `cout << obj` is `cout`, an `ostream` object, not the user's class — since `operator<<` would need to be a member of `ostream`, which can't be modified, it must be a non-member function instead.

2. Why does the overload return `ostream&` and not `void`? Give one line of code that would fail to compile if it returned `void`, and give one reason making `operator<<` a `friend` does not really weaken encapsulation.

   Returning `ostream&` lets the result chained, as in `cout << a << b;` — that line would fail to compile if it returned `void`. `friend` doesn't weaken encapsulation, because the class itself chooses to grant that one function access.

3. Members are initialized in *declaration* order, not list order. In `class Buffer { int size; int* data; Buffer(int n) : data(new int[size]), size(n) {} };` — what goes wrong, and how do you fix it?

   Members initialize in declaration order, so `size(n)` runs before `data(new int[size])` and it happens to work — but it should be rewritten to match: `Buffer(int n) : size(n), data(new int[size]) {}`

4. Give two distinct reasons to prefer a member initializer list over plain assignment in the constructor body.

   An initializer list is required for `const` / reference members, and it's more efficient for class-type members since it constructs them directly instead of default-constructing then reassigning.